



P5 - OPENSPEC

SPECIFICATION-DRIVEN DEVELOPMENT

Esta práctica es continuación de P4 – LA FÁBRICA DE SOFTWARE. En esta práctica se instala e inicializa OpenSpec en el repositorio AGENDA y se utiliza para preparar la primera especificación antes de generar código.

1 CONTEXTO

En la práctica P4 se configuró una pequeña fábrica de software utilizando GitHub. Para el proyecto AGENDA se creó un repositorio compartido, un Product Backlog, una primera historia de usuario, un panel de seguimiento y una pipeline básica.

Hasta este momento hemos definido principalmente qué trabajo queremos realizar mediante historias de usuario e Issues. En esta práctica introduciremos **Specification-Driven Development (SDD)**: antes de pedir a una IA que genere código, describiremos de forma explícita qué debe hacer el sistema.

Para ello utilizaremos OpenSpec, incorporando las especificaciones al propio repositorio y tratándolas como artefactos de ingeniería versionados, siguiendo el siguiente ciclo de vida:

```
Historia de usuario → Especificación → Diseño → Tareas →  
Implementación
```

2 OBJETIVOS

- Comprender el concepto básico de *Specification-Driven Development* (SDD).
- Instalar OpenSpec en el entorno de desarrollo.

- Inicializar OpenSpec dentro del repositorio AGENDA creado en P1.
- Identificar los artefactos incorporados por OpenSpec al repositorio.
- Relacionar una historia de usuario del backlog con una especificación.
- Utilizar un asistente de IA conjuntamente con OpenSpec.
- Revisar críticamente una especificación generada con ayuda de IA.
- Mantener las especificaciones dentro del repositorio y bajo control de versiones.

3 MODELO DE TRABAJO

En P1 utilizamos el siguiente flujo:

```
Necesidad
  ↓
Historia de Usuario
  ↓
Issue GitHub
  ↓
Sprint
  ↓
Desarrollo
```

A partir de esta práctica añadimos una capa explícita de especificación:

Principio de la práctica: no pasar directamente de una descripción en lenguaje natural al código generado por IA. La especificación debe poder ser leída, discutida, revisada, versionada y aprobada por el equipo.

4 PREPARAR EL ENTORNO

4.1 Crear el proyecto AGENDA desde GitHub en VCode

- Si no se ha hecho con antelación, clonar desde GitHub el repositorio AGENDA en el ordenador local con VCode.
- Comprobar que se está trabajando en la rama main. Abrir un terminal y ejecutar:

```
git status
```

```
PS G:\Mi unidad\12 CHILE\1 IA-IS\VCODE\BORRAR-1> git status
On branch main
Your branch is up to date with 'origin/main'.
```

El resultado debe indicar que estamos en la rama principal del repositorio y mostrar el estado actual del *working tree*.

4.2 Instalar Node.js

OpenSpec requiere Node.js. Antes de instalarlo, comprobar si ya está disponible:

```
node --version
```

Comprobar también npm:

```
npm --version
```

Si no está instalado estos comandos darán error.

Para instalar node.js

- Accede al sitio oficial de Node <https://nodejs.org/es/download>.
- Descarga la versión LTS (*Long Term Support*).
- Ejecuta el instalador
- Es importante que quede **habilitada la incorporación de Node.js al PATH**, para poder ejecutar node y npm desde cualquier terminal.
- Finaliza la instalación y cierra y vuelve a abrir Visual Studio Code.
- En VS Code abre ejecutar de nuevo los comandos anteriores

4.3 Instalar OpenSpec

- Desde la terminal de Visual Studio Code ejecutar:
- Comprobar la instalación:

```
openspec --version
```

La comprobación importante es que el comando se ejecute sin errores y devuelva una versión instalada.

4.4 Inicializar OpenSpec en proyecto AGENDA

- Comprobar que se está en la raíz del proyecto VCode “AGENDA”

```
Get-Location
```

Debe corresponder al repositorio AGENDA, por ejemplo:

```
C:\IA-IS\AGENDA
```

- Comprobar de nuevo:

```
git status
```

- Desde la raíz del repositorio AGENDA ejecutar:

```
openspec init
```

Seguir el asistente interactivo de inicialización. Si se solicitan integraciones con asistentes de IA, seleccionar GitHub Copilot.

```
Note: OpenSpec collects anonymous usage stats. Opt out: OPENSPEC_TELEMETRY=0 or openspec config set telemet
ry.enabled false

Welcome to OpenSpec
A lightweight spec-driven framework

This setup will configure:
  • Agent Skills for AI tools
  • Workflow commands, if supported

Quick start after setup:
  /opsx:propose Start a change
  /opsx:apply   Implement tasks
  (spelling varies by tool)

Press Enter to select tools...
```

Seleccionar solo GitHub Copilot de la lista de asistentes IA a instalar

- Después de ejecutar `openspec init`, no generar todavía código. Primero inspeccionar los cambios:

```
git status
```

- Desde el explorador de Visual Studio Code identificar:
 - Qué carpetas ha creado OpenSpec.
 - Qué ficheros de configuración aparecen.
 - Qué ficheros están relacionados con el asistente de IA.
 - Qué elementos deberán quedar almacenados en Git.

5 PRIMERA INTERACCIÓN CON OPENSPEC

Vamos a hacer una interacción completa de revisión de una historia de usuario y generación de especificaciones para esa historia de usuario según el siguiente flujo:

```
HU-01
↓
resolver ambigüedades
↓
actualizar criterios de aceptación
↓
/opsx:propose registrar-persona <-Instrucción OpenSpec que genera:
↓
proposal.md
↓
spec.md
↓
design.md
↓
tasks.md
↓
REVISIÓN HUMANA
```

5.1 Analizar con Copilot HI – 01 Registrar Usuario

Vamos a relacionar las historias de usuario definidas en AGENDA con OpenSpec

En P1 se definió como ejemplo la historia:

```
HU-01 Registrar persona
```

- Localizar en GitHub:

```
Repository → Issues → HU-01 Registrar persona
```

- Revisar la descripción, los criterios de aceptación, las ambigüedades y la información ausente.
- Abrir GitHub Copilot Chat en Visual Studio Code. La IA se utilizará para analizar la funcionalidad y mejorar la especificación, no para generar directamente código (de momento). NO usar prompts del tipo:

```
Programa una agenda.
Crea una aplicación CRUD.
```

El objetivo es trabajar primero sobre la especificación.

- Prompt inicial sugerido:

Estamos desarrollando una aplicación AGENDA.

Queremos implementar la historia de usuario HU-01 Registrar persona:
Como usuario propietario de la agenda quiero poder registrar nuevas personas dentro de la agenda con su nombre, apellidos, fecha de nacimiento, correo electrónico, teléfono, dirección, categoría y comentarios para posteriormente poder buscar información de personas de mi agenda.

Quiero utilizar OpenSpec y trabajar siguiendo *Specification-Driven Development*.

Analiza la funcionalidad.

Identifica ambigüedades, restricciones y decisiones que deberíamos resolver en la especificación.

No escribas todavía código.

Ver **Anexo I** de posible salida del chat Copilot al prompt

- Analizar las ambigüedades. Discutir en equipo y decidir sobre las ambigüedades y otras recomendaciones. Para cada ambigüedad:
 - Identificar el aspecto no definido.
 - Analizar las alternativas propuestas por la IA.
 - Tomar una decisión como equipo.
 - Si la decisión modifica o aclara el comportamiento esperado por el usuario, **actualizar los criterios de aceptación de la Issue HU-01.**
 - Si son detalles técnicos, se incorporarán a la especificación con OpenSpec

Nota. El proceso es como sigue: el análisis descubre ambigüedades → Si es un cambio de alto nivel que afecta al usuario se documenta en la Historia de Usuario → la decisión refina el requisito → SE lleva a OpenSpec. OpenSpec formaliza suficientemente el comportamiento para poder implementarlo.

5.2 Crear el primer cambio con OpenSpec

- En el Chat Copilot genera la primera instrucción OpenSpec

/opsx:propose registrar-persona

Implementar la historia de usuario HU-01 de la aplicación AGENDA.

Como usuario propietario de la agenda quiero registrar nuevas personas incluyendo nombre, apellidos, fecha de nacimiento, correo electrónico, teléfono, dirección, categoría y comentarios para posteriormente poder consultar su información.

Criterios acordados:

- nombre y apellidos son obligatorios;
- el correo electrónico es opcional;
- si se proporciona correo, debe tener formato válido;
- la fecha de nacimiento no puede ser futura;
- cada persona tendrá un identificador único generado por el sistema.

No implementar todavía código. Preparar únicamente la especificación, el diseño y las tareas necesarias.

- `/opsx:propose` crea un directorio del cambio y genera los artefactos necesarios antes de la implementación. En el flujo spec-driven, normalmente incluye:

```
proposal.md
specs/
design.md
tasks.md
```

- y los sitúa bajo una estructura similar a:

```
openspec/
├── changes/
│   └── registrar-persona/
│       ├── proposal.md
│       ├── specs/
│       │   └── ...
│       │       └── spec.md
│       ├── design.md
│       └── tasks.md
```

- OpenSpec documenta expresamente que `/opsx:propose` crea `openspec/changes/<change-name>/` y genera los artefactos necesarios antes de pasar a `/opsx:apply`.

```
proposal.md
specs/
```

```
design.md
tasks.md
```

- y los sitúa bajo una estructura similar a:

```
openspec/
├── changes/
│   └── registrar-persona/
│       ├── proposal.md
│       ├── specs/
│       │   └── ...
│       │       └── spec.md
│       ├── design.md
│       └── tasks.md
```

5.3 Revisar la propuesta antes de programar

El flujo para la implementación debe ser

```
ESPECIFICAR
  ↓
REVISAR
  ↓
IMPLEMENTAR
```

Se evita así el flujo directo:

```
PROMPT
  ↓
CODE
```

- Localizar y revisar los artefactos creados por OpenSpec. Comprobar especialmente:

Artefacto	Contenido
proposal.md	explica qué cambio se propone y por qué.
spec.md	contiene los requisitos y los escenarios de comportamiento.
design.md	contiene decisiones de diseño e implementación
tasks.md	contiene las tareas necesarias para realizar el cambio.

5.4 Revisar los escenarios (Casos) en spec.md

OpenSpec habrá generado “escenarios” o “casos” como criterios de aceptación detallados.

- Revisar los casos

```
## Criterios de aceptación detallados

### Caso 1: Registro exitoso con todos los campos
- Dado un usuario propietario de la agenda
- Cuando envía nombre, apellidos, fecha de nacimiento, correo electrónico válido,
teléfono, dirección, categoría y comentarios
- Entonces el sistema crea la persona, le asigna un `id` único y la guarda correctamente.
```

- Incorporar al menos dos casos detallados más

5.5 TRAZABILIDAD CON GITHUB

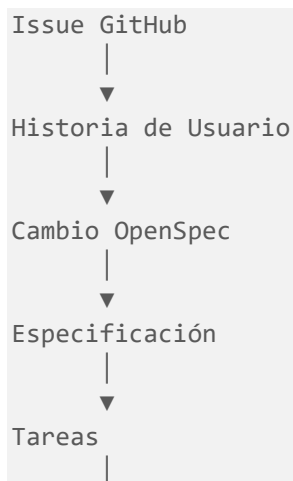
Volver a la Issue HU-01 Registrar persona y añadir un comentario indicando que la funcionalidad se está especificando mediante OpenSpec.

Ejemplo:

Esta historia está siendo desarrollada mediante SDD utilizando OpenSpec.

Cambio OpenSpec:
registrar-persona

Objetivo de trazabilidad:



▼
Código

6 ACCIONES DE FINALIZACIÓN

6.1 Actualizar los cambios en GitHub

- Hacer un push de los cambios del proyecto VCode

```
git status
git diff
git add .
git commit -m "configura AGENDA con OpenSpec para desarrollo SDD"
git push
```

Comprobar en GitHub que los nuevos artefactos aparecen en el repositorio.

6.2 Comprobar pipelines

En GitHub ir a Actions y comprobar:

- Ejecución correcta del Pipeline CI.yml generado en la práctica anterior
- Ejecución correcta de los pipelines generados por OpenSpec

HU → especificación → revisión → implementación

¿Qué enfoque proporciona mayor control al equipo y por qué?

7 BIBLIOGRAFÍA PARA LA PRÁCTICA

- Fission AI. (2026). OpenSpec: Spec-Driven Development for AI coding assistants.. <https://github.com/Fission-AI/openspec>
- Fission AI. (2026). OpenSpec Insgtallation and CLI Reference. <https://github.com/Fission-AI/OpenSpec/blob/main/docs/installation.md>
- Node.js. (2026). Node.js Documentation and Downloads. Documentación oficial para instalación de Node.js, entorno necesario para instalar y ejecutar la CLI de OpenSpec. <https://nodejs.org/en/download>

8 ANEXOS

8.1 Recomendaciones OpenSpec

Ejemplo de salida a la petición de mejora de la HI - 01

Ambigüedades a resolver en la especificación

1. Formato y validación de datos
 - ¿Cuál es el formato válido para fecha de nacimiento? (Ej. YYYY-MM-DD)
 - ¿Qué reglas se aplican al correo electrónico? (validación de formato estándar)
 - ¿qué caracteres o longitud se aceptan para nombre, apellidos, dirección y comentarios?
 - ¿el teléfono acepta solo números, separadores, código de país, extensiones?
2. Campos obligatorios vs opcionales
 - ¿todos los campos son obligatorios?
 - ¿pueden quedar en blanco comentarios o categoría?
 - ¿es obligatorio el correo electrónico o el teléfono?
3. Categoría
 - ¿qué valores puede tener categoría?
 - ¿es un campo libre de texto o un conjunto cerrado de opciones?
 - ¿hay categorías predeterminadas (familia, trabajo, amigos, otros)?
4. Duplicados
 - ¿cómo se detecta una persona ya registrada?
 - ¿se permite registrar personas con el mismo nombre y apellidos?
 - ¿se usa correo electrónico o teléfono como identificador único?
5. Búsqueda
 - ¿qué campos se pueden buscar luego?
 - ¿la búsqueda es exacta, parcial o por coincidencia?
 - ¿se puede buscar por combinación de campos?
 - ¿qué comportamiento se espera si hay múltiples resultados?
6. Alcance de la historia
 - ¿HU-01 solo cubre el registro, o también incluye validación y guardado persistente?
 - ¿la búsqueda se especifica en otra historia o forma parte de la misma funcionalidad?

Restricciones que deberían quedar claras

1. Persistencia de datos
 - ¿dónde se guarda la persona? (base de datos, almacenamiento local, archivo)
 - ¿debe ser inmediatamente disponible para búsqueda?
2. Consistencia de datos
 - ¿hay reglas sobre normalización de nombres, mayúsculas/minúsculas y espacios?

- ¿se deben sanitizar los campos para evitar inyecciones o caracteres inválidos?
- 3. Interfaces de usuario / API
 - ¿cómo se captura el registro? (formulario, endpoint API, consola)
 - ¿qué mensajes se muestran en caso de error de validación?
- 4. Requisitos de búsqueda
 - ¿la búsqueda necesita soporte para filtros por categoría o rango de edad?
 - ¿debe poder buscar por texto libre en comentarios/dirección?

Decisiones que conviene definir en la especificación

1. Tipos de datos y validaciones exactas por campo.
2. Lista de campos obligatorios y opcionales.
3. Definición de categoría como lista cerrada o texto libre.
4. Política de duplicados y criterio de unicidad.
5. Nivel de detalle de la búsqueda posterior y sus casos de uso principales.
6. Alcance de HU-01 frente a otras historias de búsqueda o edición.

Recomendación para *Specification-Driven Development*

- Documentar primero la “especificación del registro de persona” en [specs](#).
- Incluir criterios de aceptación claros para:
 - registro exitoso
 - errores de validación
 - manejo de duplicados
- Definir los campos y las reglas antes de implementar.

Nota: ahora no escribimos código. Lo siguiente sería convertir estas decisiones en especificaciones de OpenSpec y luego implementar con base en ellas.

8.2 Errores frecuentes

Errores frecuentes al realizar la práctica

- Instalar OpenSpec y no inicializarlo en AGENDA.
- Ejecutar openspec init fuera del repositorio.
- Pedir directamente a Copilot que genere el CRUD.
- Aceptar automáticamente la especificación propuesta por la IA.
- Confundir historia de usuario con especificación.
- Crear una especificación sin escenarios verificables.

- Modificar la especificación pero no almacenarla en Git.
- Generar código antes de revisar la especificación.